**Politechnika Wrocławska**
Wydział Informatyki i Telekomunikacji

**Kierunek:** Informatyczne Systemy Automatyki (ISA)
**Specjalność:** Inteligentne Systemy Przemysłu 4.0 (IPS)

# PRACA DYPLOMOWA INŻYNIERSKA

## Aplikacja wspomagająca harmonogramowanie produkcji dla wybranego zagadnienia szeregowania zadań

Bartłomiej Adam Kuk

Opiekun pracy
dr inż. Jarosław Rudy

Słowa kluczowe: harmonogramowanie, produkcja, aplikacja, optymalizacja

WROCŁAW (2025)

## Streszczenie

## Abstract

# Spis treści


**1 Powiązane prace** **1**

1.1 Integrating Sequence-Dependent Setup Times and Blocking in Hybrid Flow Shop Scheduling to Minimize Total Tardiness — Kurt (2025) . . . 1

1.2 Hybrid Flow Shop Scheduling through Reinforcement Learning — Pugliese, Romano, Lin, Zhu, Tan (2025) . . . 1

1.3 A Real Time Human Machine Logistics Collaborative Scheduling Method Considering Workers' Learning and Forgetting Effects — Yang, Li, Luo, Li, Wen (2025) . . . 2

1.4 MRLM: A meta-reinforcement learning-based metaheuristic for hybrid flow-shop scheduling problem with learning and forgetting effects — Zhang, Shao, Shao, Chen, Pi (2024) . . . 2

**2 Założenia projektu i opis problemu** **5**

2.1 Opis problemu szeregowania zadań . . . 5

2.2 Metody optymalizacji problemu HFS . . . 6

2.2.1 Algorytm dokładny – Branch and Bound (B&B) . . . 7

2.2.2 Algorytm zachłanny – Min Setup Time First . . . 8

2.2.3 Algorytm metaheurystyczny – Tabu Search . . . 9

2.3 Założenia aplikacji . . . 12

**3 Projektowanie rozwiązania** **13**

3.1 Architektura systemu . . . 13

3.1.1 Model architektury . . . 13

3.1.2 Wybrane technologie . . . 14

3.2 Opis algorytmów . . . 15

3.2.1 Algorytm dokładny – Branch and Bound (B&B) . . . 15

3.2.2 Algorytm metaheurystyczny – Tabu Search . . . 16

3.3 Struktura danych wejściowych i wyników optymalizacji . . . 18

3.4 Projekt Graficznego Interfejsu Użytkownika (GUI) . . . 21

**4 Implementacja** **25**

4.1 Silnik optymalizacji . . . 25

# 1. Powiązane prace

## 1.1. Integrating Sequence-Dependent Setup Times and Blocking in Hybrid Flow Shop Scheduling to Minimize Total Tardiness — Kurt (2025)

[3]

Autor podejmuje problem harmonogramowania w środowisku Hybrid Flow Shop, w którym uwzględnia dwa istotne aspekty wpływające na realizację zadań: czasy przezbrojenia zależne od kolejności operacji (sequence-dependent setup times) oraz brak buforów pomiędzy kolejnymi etapami produkcji (blocking). Głównym celem optymalizacji jest minimalizacja całkowitego opóźnienia zadań (total tardiness), co stanowi alternatywę dla najczęściej spotykanego kryterium minimalizacji czasu zakończenia (makespan).

W pracy przedstawiono dwa modele dokładne oparte na programowaniu całkowitoliczbowym (MILP), które umożliwiają rozwiązywanie małych instancji problemu. Dla większych instancji zaproponowano podejścia heurystyczne: algorytm lokalnego przeszukiwania (ILS) oraz hybrydowy algorytm genetyczny (HGA). Wyniki eksperymentów wykazały, że metody heurystyczne pozwalają na uzyskanie wysokiej jakości rozwiązań w krótkim czasie obliczeniowym.

Mimo że autor nie uwzględnia w modelu efektu uczenia się, praca dostarcza wartościowych narzędzi i wniosków w kontekście integracji przezbrojeń i blokowania w harmonogramowaniu produkcji typu HFS. Może stanowić punkt odniesienia dla projektowania rozwiązań metaheurystycznych w bardziej złożonych modelach uwzględniających dodatkowe zjawiska dynamiczne.

## 1.2. Hybrid Flow Shop Scheduling through Reinforcement Learning — Pugliese, Romano, Lin, Zhu, Tan (2025)

[5]

Autorzy przedstawiają systematyczny przegląd literatury dotyczący zastosowania metod uczenia przez wzmacnianie (Reinforcement Learning, RL) w problemach harmonogramowania typu Hybrid Flow Shop (HFS), ze szczególnym uwzględnieniem rozszerzeń obejmujących efekt uczenia się, zapominania oraz czasy przezbrojeń. W pracy dokonano klasyfikacji istniejących badań pod względem stosowanych algorytmów RL (m.in. Q-learning, PPO, DDPG), reprezentacji stanów, funkcji nagrody oraz strategii eksploracji.

Przegląd pokazuje, że większość istniejących prac koncentruje się na uproszczonych wersjach HFS, natomiast modele zawierające jednocześnie efekty uczenia, zapominania i setup times są rzadkością. Autorzy identyfikują lukę badawczą w postaci braku kompleksowych podejść

integrujących te trzy zjawiska w jednym środowisku RL. Wskazano również na rosnącą skuteczność podejść hybrydowych, łączących RL z metaheurystykami, takimi jak algorytmy genetyczne czy symulowane wyżarzanie.

Wnioski zawarte w pracy stanowią cenne źródło wiedzy dla dalszych badań nad RL w kontekście rzeczywistych systemów produkcyjnych, w których istotna jest adaptacja do dynamicznych warunków oraz zdolność do modelowania efektów zależnych od historii operacji.

## 1.3. A Real Time Human Machine Logistics Collaborative Scheduling Method Considering Workers' Learning and Forgetting Effects — Yang, Li, Luo, Li, Wen (2025)

[7]

Autorzy proponują innowacyjne podejście do harmonogramowania w czasie rzeczywistym w inteligentnych warsztatach, które integruje w jednym modelu zasoby ludzkie, maszynowe i logistyczne. Kluczowym elementem jest dynamiczny model efektywności pracowników, oparty na efektach uczenia i zapominania, pozwalający na logistyczno-produkcyjne grupowanie zasobów oraz adaptacyjne dopasowanie zadań do dostępnych grup. Badania eksperymentalne potwierdzają, że rozważenie tych efektów prowadzi do znaczącej redukcji czasu zakończenia zadań, opóźnień i zużycia energii.

Metodologia obejmuje model matematyczny opisujący efekty uczenia i zapominania, algorytm dopasowujący zadania do grup zasobów na podstawie luzu czasowego oraz mechanizm samoorganizacji zespołów w czasie rzeczywistym. Praca stanowi wartościowe odniesienie przy badaniu metod opartych na uczeniu ze wzmocnieniem lub metaheurystykach w kontekście harmonogramowania z efektami uczenia i zapominania w zmiennym środowisku produkcyjnym.

## 1.4. MRLM: A meta-reinforcement learning-based metaheuristic for hybrid flow-shop scheduling problem with learning and forgetting effects — Zhang, Shao, Shao, Chen, Pi (2024)

[9]

Autorzy proponują innowacyjną metodę opartą na meta-uczeniu ze wzmocnieniem (meta-reinforcement learning), która służy do rozwiązywania rozszerzonego Hybrydowego Problemu Przepływu Zadań (HFSP), z uwzględnieniem efektu uczenia się i zapominania. Artykuł przedstawia model MRLM (Meta-Reinforcement Learning-based Metaheuristic), który adaptuje się do różnych instancji problemu oraz dynamicznych zmian w systemie produkcyjnym. W modelu zastosowano architekturę Actor-Critic oraz technikę task-specific embeddingów.

W eksperymentach porównawczych z klasycznymi metaheurystykami (np. GA, SA, TS) zaproponowana metoda wykazuje lepszą zdolność generalizacji oraz krótszy czas znajdowania wysokiej jakości rozwiązań. Wyniki sugerują, że podejście oparte na meta-uczeniu może skutecznie wspierać harmonogramowanie w zmiennych środowiskach przemysłowych.

## Podsumowanie przeglądu literatury

Przegląd literatury dotyczy nowoczesnych metod rozwiązywania problemu harmonogramowania w środowisku Hybrid Flow Shop (HFS), ze szczególnym uwzględnieniem czasów przezbrojeń, efektów uczenia się i zapominania oraz zastosowań sztucznej inteligencji, w tym uczenia przez wzmacnianie (Reinforcement Learning, RL) i metaheurystyk.

**Kurt (2025)** analizuje HFS z przezbrojeniami zależnymi od kolejności i blokowaniem, proponując modele MILP oraz heurystyki (ILS, HGA). **Pugliese i in. (2025)** dokonują przeglądu metod RL, wskazując brak podejść uwzględniających jednocześnie uczenie, zapominanie i przezbrojenia. **Yang i in. (2025)** modelują dynamiczną efektywność pracowników w czasie rzeczywistym, a **Zhang i in. (2024)** proponują zaawansowaną metaheurystykę MRLM opartą na meta-RL.

**Wnioski:** Omówione prace ukazują *ewolucję podejść* do problemu harmonogramowania HFS: od klasycznych modeli optymalizacyjnych, przez zastosowanie heurystyk, aż po inteligentne systemy oparte na uczeniu maszynowym. Integracja *efektów uczenia się, zapominania i przezbrojeń* nadal stanowi wyzwanie badawcze, jednak nowoczesne rozwiązania hybrydowe oparte na RL i metaheurystykach wskazują obiecujący kierunek dalszych badań.

# 2. Założenia projektu i opis problemu

## 2.1. Opis problemu szeregowania zadań

Rozważany jest problem szeregowania zadań w środowisku produkcyjnym typu *hybrydowy przepływowy* (Hybrid Flow Shop – HFS), w którym występuje $s$ etapów (faz), a każdy etap $a \in \{1, \ldots, s\}$ dysponuje określoną liczbą $m_a$ maszyn pracujących równolegle, gdzie $m_a \geq 1$, a istnieje przynajmniej jeden etap $a$ taki, że $m_a \geq 2$. Celem jest zaplanowanie przetwarzania $n$ zadań na dostępnych zasobach w taki sposób, aby uwzględnić ograniczenia technologiczne oraz zoptymalizować przyjęte kryterium jakości harmonogramu.

Każde zadanie $J_j \in \{J_1, \ldots, J_n\}$ składa się z $s$ operacji $O_{j1}, O_{j2}, \ldots, O_{js}$, które muszą zostać wykonane kolejno, tj. operacja $O_{ja}$ musi zakończyć się przed rozpoczęciem $O_{j(a+1)}$. Operacja $O_{ja}$ może zostać przypisana do jednej z dostępnych maszyn na etapie $a$, przy czym wybór maszyny oraz kolejność operacji wpływa na ostateczny harmonogram.

### Elementy modelu

- **Liczba maszyn:** Każdy etap $a \in \{1, \ldots, s\}$ dysponuje $m_a$ maszynami, które pracują równolegle i mogą niezależnie wykonywać operacje przypisane do tego etapu. Zbiór liczby maszyn na wszystkich etapach oznaczamy jako $M = \{m_1, m_2, \ldots, m_s\}$ i traktujemy jako konfigurację zasobów produkcyjnych systemu.

- **Czasy przetwarzania:** Czas bazowy realizacji operacji $O_{ja}$ przez maszynę $k$ oznaczany jest jako $p_{jak} > 0$.

- **Przezbrojenia zależne od kolejności (SDST):** Po wykonaniu operacji $O_{ja}$ przypisanej do maszyny $k$, przejście do kolejnej operacji $O_{j'a}$ może wymagać przezbrojenia. Czas przezbrojenia zależy od pary kolejnych zadań i oznaczany jest jako $s_{jj'}^{ak}$.

- **Efekt uczenia się:** Zakłada się, że czas realizacji operacji skraca się w miarę wykonywania kolejnych zadań dzięki akumulacji doświadczenia. Zmodyfikowany czas przetwarzania uwzględniający efekt uczenia opisuje wzór:

  $$p_{jak}^{(r)} = p_{jak} \cdot r^{-\alpha} \tag{2.1}$$

  gdzie:

  - $r$ – liczba operacji już zrealizowanych na maszynie $k$ przed bieżącą operacją,
  - $\alpha > 0$ – współczynnik uczenia (większa wartość oznacza silniejszy efekt),
  - $p_{jak}$ – bazowy czas operacji bez efektu uczenia.

  Efekt ten kumuluje się lokalnie – osobno na każdej maszynie – i jest niezależny od typu realizowanego zadania.

### Założenia i ograniczenia

1. Każde zadanie musi przejść przez wszystkie etapy w ustalonej kolejności.
2. Każda maszyna może jednocześnie przetwarzać tylko jedną operację.
3. Każda operacja może być wykonana tylko raz i tylko przez jedną maszynę danego etapu.
4. Przezbrojenia są wymagane między kolejnymi operacjami na tej samej maszynie.
5. Efekt uczenia się działa lokalnie dla każdej maszyny – niezależnie od typu zadania – i nie zanika w czasie.

### Funkcja celu

Celem optymalizacji jest minimalizacja czasu zakończenia realizacji ostatniego zadania w harmonogramie, określanego jako *makespan*. Formalnie:

$$C_{\max} = \max_{j \in \{1,\ldots,n\}} C_j \tag{2.2}$$

gdzie $C_j$ to czas zakończenia ostatniej operacji zadania $j$. W związku z tym funkcja celu przyjmuje postać:

$$\min C_{\max} \tag{2.3}$$

Minimalizacja wartości $C_{\max}$ pozwala na skrócenie całkowitego czasu realizacji wszystkich zadań w systemie.

## 2.2. Metody optymalizacji problemu HFS

W celu rozwiązania problemu harmonogramowania zadań w środowisku Hybrid Flow Shop z przezbrojeniami zależnymi od kolejności oraz efektem uczenia, konieczne było zastosowanie odpowiednio dobranych algorytmów optymalizacyjnych. Ze względu na złożoność obliczeniową zagadnienia, obejmującą cechy problemów klasy NP-trudnych, w projekcie wykorzystano zarówno podejścia heurystyczne, jak i metaheurystyczne.

W niniejszej sekcji przedstawiono przegląd zastosowanych metod, podzielonych na trzy główne kategorie:

- **Algorytmy dokładne** – wykorzystujące deterministyczne podejście do poszukiwania rozwiązania optymalnego.
- **Algorytmy zachłanne (heurystyki)** – szybkie metody konstrukcyjne, generujące rozwiązania początkowe na podstawie lokalnych reguł priorytetów.
- **Algorytmy metaheurystyczne** – strategie inspirowane zjawiskami naturalnymi, umożliwiające eksplorację przestrzeni rozwiązań w sposób probabilistyczny.

### 2.2.1. Algorytm dokładny – Branch and Bound (B&B)

Algorytm *Branch and Bound* (B&B) jest metodą dokładną wykorzystywaną do znajdowania rozwiązania optymalnego poprzez systematyczne przeszukiwanie przestrzeni wszystkich możliwych permutacji zadań. W kontekście problemu *Hybrid Flow Shop* (HFS) z przezbrojeniami zależnymi od kolejności oraz efektem uczenia się, metoda ta pozwala uwzględnić dodatkowe ograniczenia technologiczne podczas wyznaczania harmonogramu minimalizującego $C_{\max}$.

W najgorszym przypadku złożoność obliczeniowa algorytmu wynosi $O(n!)$, ponieważ rozważane są wszystkie możliwe kolejności zadań. W praktyce jednak stosowane są mechanizmy ograniczające (*bounds*), które umożliwiają odrzucanie (*pruning*) nieperspektywicznych gałęzi drzewa przeszukiwań, co pozwala znacząco zmniejszyć liczbę analizowanych rozwiązań w porównaniu do przeglądu zupełnego.

Algorytm opiera się na dwóch podstawowych typach ograniczeń:

A. **Dolne ograniczenie (Lower Bound, LB)** – określa minimalny możliwy koszt (czas zakończenia) dla częściowego rozwiązania. Pozwala odrzucać gałęzie drzewa, które nie mogą prowadzić do lepszego rozwiązania niż aktualnie znane.

B. **Górne ograniczenie (Upper Bound, UB)** – stanowi oszacowanie maksymalnego kosztu harmonogramu dla danej częściowej permutacji, co przyspiesza odrzucanie rozwiązań gorszych od najlepszego dotychczas znalezionego.

Algorytm przyjmuje jako dane wejściowe zbiór $n$ zadań, macierze czasów operacji $p_{jak}$, czasy przezbrojeń $s_{jj'}^{ak}$ oraz współczynnik uczenia $\alpha$. Wynikiem działania jest optymalna permutacja $\pi^*$ minimalizująca $C_{\max}$ wraz z odpowiadającym jej harmonogramem. Dzięki zastosowaniu ograniczeń $LB(\pi')$ i $UB(\pi')$, algorytm może efektywnie odrzucać nieoptymalne gałęzie drzewa, skracając czas działania przy zachowaniu gwarancji optymalności.

**Algorithm 2.1:** Branch and Bound dla HFS z przezbrojeniami i efektem uczenia

**Input:** Zbiór $n$ zadań $J = \{J_1, J_2, \ldots, J_n\}$, macierze czasów operacji $p_{jak}$, czasy przezbrojeń $s^{ak}_{jj'}$, współczynnik uczenia $\alpha$

**Output:** Optymalna permutacja $\pi^*$ minimalizująca $C_{\max}$

```
π* ← ∅ ;                                   // najlepsza znaleziona permutacja
C* ← ∞ ;                                   // koszt najlepszego rozwiązania
Q ← kolejka z pustą permutacją π
while Q niepusta do
    π ← Q.pop()
    if |π| = n then
        if C_max(π) < C* then
            C* ← C_max(π)
            π* ← π
    else
        foreach j ∈ zadania nieobecne w π do
            π' ← π ∘ j ;                   // dodaj zadanie j na koniec permutacji
            if LB(π') < C* then
                if UB(π') < C* then
                    Q.push(π') ;           // rozwiń tylko obiecujące gałęzie
return π*
```

W algorytmie:

- $\pi$ – reprezentuje częściową lub pełną permutację zadań,
- $LB(\pi')$ – dolne oszacowanie kosztu harmonogramu dla częściowej permutacji $\pi'$,
- $C_{\max}(\pi)$ – całkowity czas realizacji zadań dla permutacji $\pi$,
- $UB(\pi')$ – górne oszacowanie kosztu dla danej częściowej permutacji.

Metoda Branch and Bound, mimo wysokiej złożoności obliczeniowej, stanowi punkt odniesienia dla metod przybliżonych i metaheurystycznych. W badaniach nad problemem *Hybrid Flow Shop* wykorzystywana jest często jako algorytm porównawczy do oceny jakości rozwiązań generowanych przez heurystyki i metaheurystyki [2, 4].

### 2.2.2. Algorytm zachłanny – Min Setup Time First

Algorytm zachłanny *Min Setup Time First* (MSTF lub MinSTF) jest prostą, lecz skuteczną heurystyką wykorzystywaną w problemach szeregowania z przezbrojeniami zależnymi od kolejności (SDST). W każdej iteracji wybierane jest kolejne zadanie, które powoduje **najmniejszy dodatkowy koszt przezbrojenia** względem ostatnio zaplanowanego zadania. Dzięki temu metoda minimalizuje lokalne koszty zmian konfiguracji maszyn, tworząc kolejność zadań w sposób szybki i intuicyjny.

Metoda MSTF nie gwarantuje globalnej optymalności rozwiązania (w przeciwieństwie do algorytmów dokładnych, takich jak metoda Branch and Bound o złożoności $O(n!)$), lecz umożliwia w czasie wielomianowym, zwykle $O(n^2)$, uzyskanie harmonogramów o akceptowalnej jakości. Z tego względu często stosowana jest jako procedura inicjalizująca dla bardziej złożonych metod metaheurystycznych.

**Algorithm 2.2:** Min Setup Time First dla HFS-SDST z efektem uczenia

**Input:** Zbiór $n$ zadań $J = \{J_1, J_2, \ldots, J_n\}$, macierze czasów operacji $p_{jak}$, czasy przezbrojeń $s^{ak}_{jj'}$, współczynnik uczenia $\alpha$

**Output:** Permutacja $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$ minimalizująca $C_{\max}$

```
π ← ∅ ;                                                  // inicjalizacja permutacji
J_reszta ← J ;                                           // zadania do zaplanowania
J_poprzednie ← zadanie startowe (np. zerowe lub fikcyjne)
while J_reszta ≠ ∅ do
    foreach J_i ∈ J_reszta do
        Oblicz efektywny czas przezbrojenia s^eff_{J_poprzednie,i} z uwzględnieniem α
    Wybierz J_i z minimalnym s^eff_{J_poprzednie,i}
    π ← π ∘ J_i ;                                        // dodaj zadanie J_i do permutacji
    J_poprzednie ← J_i
    J_reszta ← J_reszta \ {J_i}
return π
```

W kontekście problemu typu *Hybrid Flow Shop* (HFS) z przezbrojeniami zależnymi od kolejności oraz efektem uczenia się, algorytm może uwzględniać fakt, że zarówno czasy przezbrojeń, jak i czasy operacji ulegają skróceniu wraz z kolejnymi realizowanymi zadaniami na tej samej maszynie. Tego typu heurystyki konstrukcyjne zostały z powodzeniem zastosowane m.in. w badaniach dotyczących problemów HFS z efektami uczenia i przezbrojeniami, gdzie wykazano, że reguły lokalnej minimalizacji czasów przezbrojeń (*MSTF-like rules*) pozwalają znacząco skrócić czas obliczeń przy zachowaniu wysokiej jakości rozwiązań.

Algorytm działa w czasie wielomianowym — każda z $n$ iteracji wymaga przeglądu co najwyżej $n - i$ pozostałych zadań, co prowadzi do złożoności czasowej rzędu $O(n^2)$. W porównaniu z algorytmami dokładnymi (takimi jak *Branch and Bound*), które wymagają przeszukania całej przestrzeni permutacji o złożoności $O(n!)$, metoda MSTF zapewnia znacznie szybsze działanie przy niewielkim spadku jakości rozwiązania. Skuteczność tego podejścia została potwierdzona wynikami badań w literaturze [6, 8].

### 2.2.3. Algorytm metaheurystyczny – Tabu Search

Kompromisem pomiędzy dokładnością a czasem obliczeń są algorytmy metaheurystyczne, które umożliwiają uzyskanie rozwiązań o wysokiej jakości w rozsądnym czasie obliczeniowym. Przykładem takiego podejścia jest algorytm Tabu Search, opierający się na probabilistycznym przeszukiwaniu przestrzeni rozwiązań w celu uniknięcia lokalnych minimów oraz zwiększenia efektywności procesu optymalizacji.

Algorytm ten w sposób iteracyjny przeszukuje sąsiedztwo aktualnego rozwiązania, dążąc do znalezienia lepszego wariantu w jego otoczeniu. Aby zapobiec powracaniu do wcześniej

odwiedzonych rozwiązań i uniknąć cyklicznego przeszukiwania tych samych obszarów, stosuje się listę ruchów zabronionych (Tabu List). Długość tej listy może być określona statycznie lub dostosowywana dynamicznie w zależności od przyjętej strategii działania algorytmu. W pewnych sytuacjach możliwe jest jednak złamanie zakazu tabu – jeśli dany ruch prowadzi do rozwiązania lepszego niż dotychczas najlepsze znalezione. Takie wyjątki są dopuszczane na podstawie tzw. kryterium aspiracji, które zwiększa elastyczność algorytmu i pozwala uniknąć pominięcia potencjalnie optymalnych rozwiązań.

Definicja sąsiedztwa może przyjmować różne formy — od prostych operacji zamiany dwóch zadań w harmonogramie po bardziej złożone manipulacje, takie jak obrót lub przesunięcie podciągu permutacji. Pozwala to na eksplorację różnych regionów przestrzeni rozwiązań i zwiększa szansę na znalezienie rozwiązania bliskiego optymalnemu.

Złożoność obliczeniowa algorytmu zależy od wielu czynników, takich jak liczba iteracji, długość listy tabu, sposób definiowania sąsiedztwa oraz przede wszystkim liczba zadań w harmonogramie. Pomimo wpływu tych parametrów, algorytm Tabu Search zachowuje złożoność wielomianową, co sprawia, że jest istotnie szybszy od metod dokładnych, przy jednoczesnym zachowaniu wysokiej jakości otrzymywanych rozwiązań.

**Algorithm 2.3:** Klasyczny algorytm Tabu Search dla problemu HFS z przezbrojeniami i efektem uczenia

**Input:** Zbiór $n$ zadań, macierze czasów operacji $p_{jak}$, czasy przezbrojeń $s^{ak}_{jj'}$, współczynnik uczenia $\alpha$

**Output:** Permutacja $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$ minimalizująca $C_{\max}$

```
π ← permutacja początkowa (np. wygenerowana heurystycznie);
π* ← π ;                                   // najlepsze dotąd znalezione rozwiązanie
tabuList ← ∅;
for iter = 1 to maxIter do
    sąsiedzi ← zbiór rozwiązań powstałych w wyniku zastosowania wybranej operacji
      sąsiedztwa do π;
    wybierz π' z sąsiedztwa o najmniejszym C_max, nieuwzględnione w tabuList lub
      spełniające kryterium aspiracji;
    if C_max(π') < C_max(π*) then
        π* ← π';
    zaktualizuj tabuList (dodaj wykonany ruch, usuń najstarsze wpisy);
    π ← π';
return π*
```

Algorytm Tabu Search jest jedną z najczęściej stosowanych metaheurystyk w problemach harmonogramowania o dużej złożoności obliczeniowej. Dzięki wykorzystaniu pamięci krótkoterminowej (listy tabu) oraz mechanizmów kontrolnych, takich jak kryterium aspiracji, umożliwia efektywne przeszukiwanie przestrzeni rozwiązań i unikanie lokalnych minimów. Jego elastyczność pozwala na dostosowanie struktury sąsiedztwa, długości listy tabu oraz kryteriów zatrzymania do specyfiki danego problemu. W literaturze algorytm ten wykazał wysoką skuteczność w rozwiązywaniu problemów typu Hybrid Flow Shop z dodatkowymi ograniczeniami, takimi jak czasy przezbrojeń, brak przestojów czy efekt uczenia się, przewyższając często metody dokładne pod względem relacji jakość–czas obliczeń [1].

# Podsumowanie metod optymalizacji

W rozdziale przedstawiono różne podejścia do rozwiązywania problemu harmonogramowania w środowisku Hybrid Flow Shop z przezbrojeniami zależnymi od kolejności oraz efektem uczenia. Omówione metody obejmowały:

- algorytm dokładny (Branch and Bound), gwarantujący rozwiązanie optymalne, lecz nieefektywny obliczeniowo dla dużych instancji,
- algorytmy zachłanne (np. Min Setup Time First), które zapewniają szybkie, choć niekoniecznie optymalne harmonogramy
- metaheurystyki (np. Tabu Search), stanowiące kompromis między jakością rozwiązania a czasem jego wyznaczania.

Każde z tych podejść znajduje zastosowanie zależnie od rozmiaru instancji, dostępnego czasu obliczeniowego oraz potrzeby co do jakości harmonogramu.

## 2.3. Założenia aplikacji

Projektowana aplikacja desktopowa ma stanowić narzędzie wspomagające proces optymalizacji harmonogramu produkcji. Jej głównym celem jest umożliwienie użytkownikowi łatwego definiowania danych wejściowych, uruchamiania algorytmu optymalizacyjnego oraz prezentacji uzyskanego harmonogramu w czytelnej formie graficznej.

### Wymagania funkcjonalne

Aplikacja powinna umożliwiać:

1. wczytywanie danych wejściowych z pliku (np. w formacie JSON),
2. ręczne definiowanie parametrów zadania i instancji problemu,
3. wybór algorytmu optymalizacyjnego: aplikacja powinna umożliwiać uruchomienie co najmniej jednego algorytmu z każdego z trzech podejść:
   (a) Podejście zachłanne
   (b) Podejście dokładne
   (c) Podejście metaheurystyczne
4. uruchomienie algorytmu i wyświetlenie wyników na ekranie,
5. wizualizację harmonogramu w formie wykresu Gantta,
6. eksport wyników do pliku (np. w formacie JSON lub jako obraz JPG).

### Wymagania niefunkcjonalne

Aplikacja powinna spełniać następujące wymagania pozafunkcjonalne:

1. zapewniać czytelny i intuicyjny interfejs użytkownika,
2. gwarantować krótkie czasy odpowiedzi dla standardowych rozmiarów danych,
3. działać w trybie offline, bez konieczności dostępu do Internetu.

# 3. Projektowanie rozwiązania

## 3.1. Architektura systemu

Architektura systemu została zaplanowana tak, aby zapewnić modularność, czytelność struktury kodu oraz możliwość przyszłej rozbudowy. System będzie składał się z dwóch głównych komponentów: części obliczeniowej (Python) oraz interfejsu użytkownika (JavaFX).

### 3.1.1. Model architektury

System zostanie zaimplementowany w architekturze warstwowej (ang. layered architecture), składającej się z następujących głównych warstw:

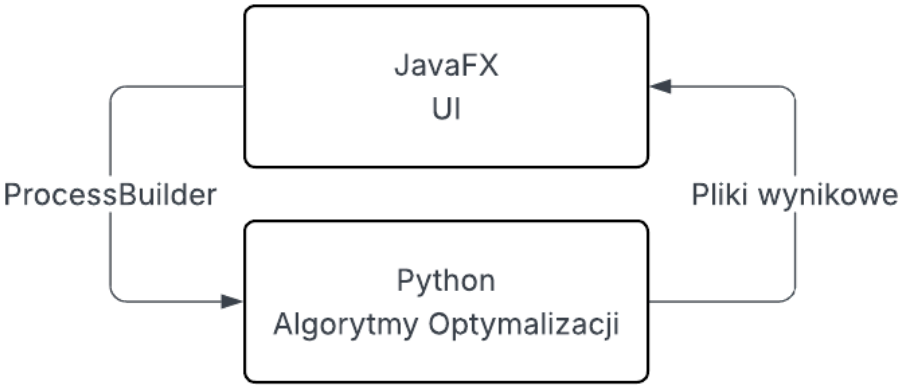
- **Warstwa prezentacji (UI)** Zaimplementowana w technologii JavaFX. Odpowiada za interakcję z użytkownikiem, w tym wczytanie danych, wybór algorytmu, konfigurację parametrów oraz prezentacje wyników.
- **Warstwa komunikacji (uruchamianie procesu)** Odpowiada za przekazywanie danych między interfejsem a częścią obliczeniową. Wykorzystuje mechanizm `ProcessBuilder` do uruchamiania skryptów Pythona bezpośrednio z poziomu aplikacji Java, przekazując dane w formacie JSON oraz odczytując wynik przetwarzania ze strumienia wyjściowego.
- **Warstwa logiki dziedzinowej (silnik optymalizacji)** Zaimplementowane w Pythonie. Odpowiada za uruchamianie algorytmów optymalizacyjnych, przetwarzanie danych wejściowych i generowanie harmonogramu. Każdy algorytm stanowi osobny moduł, co pozwala na łatwe porównywanie i testowanie.

### 3.1.2. Wybrane technologie

- **Część obliczeniowa (silnik):** Python [TO DO] - wpisać wersje po napisaniu programu
- **Interfejs graficzny (UI):** JavaFX [TO DO] - wpisać wersje po napisaniu programu
- **Komunikacja między warstwami:** `ProcessBuilder` (uruchamianie Pythona z poziomu Javy)
- **Wizualizacja:** Wykres Gantta generowany w Pythonie i wyświetlany w JavaFX jako gotowy obraz
- **Zapis/odczyt danych:** Pliki JSON (dla danych wejściowych/wyjściowych)

### Diagram komunikacji między komponentami

Diagram poniżej przedstawia uproszczony przepływ danych i komunikację pomiędzy głównymi komponentami systemu.




Rysunek 3.1: Diagram komunikacji pomiędzy JavaFX a modułem optymalizacji w Pythonie

## 3.2. Opis algorytmów

W tej części pracy przedstawiono szczegółowy opis zastosowanych algorytmów optymalizacyjnych oraz ich elementów składowych, wykorzystanych w procesie rozwiązywania problemu harmonogramowania w środowisku typu *Hybrid Flow Shop*. Opis obejmuje konkretne rozwiązania implementacyjne, takie jak sposób wyznaczania dolnych i górnych ograniczeń (LB i UB), przyjęte reguły sąsiedztwa, kryteria aspiracji, strukturę listy tabu oraz funkcje oceny harmonogramów. W odróżnieniu od części teoretycznej, niniejszy rozdział koncentruje się na praktycznej realizacji i parametryzacji poszczególnych algorytmów, uwzględniającej specyfikę analizowanego problemu.

### 3.2.1. Algorytm dokładny – Branch and Bound (B&B)

#### Dolne ograniczenie (LB)

Dolne ograniczenie określa minimalny możliwy czas zakończenia produkcji dla częściowego rozwiązania $\pi'$. W zastosowanej implementacji $LB$ zostało zdefiniowane jako maksymalne obciążenie dowolnej maszyny w bieżącym węźle drzewa, przy uwzględnieniu dotychczas przydzielonych operacji oraz czasów przezbrojeń:

$$LB(\pi') = \max_{i=1..m} \left( \sum_{J_j \in M_i} (p_j + s_{prev(j),j}) \right) \tag{3.1}$$

gdzie:

- $p_j$ – czas przetwarzania zadania $J_j$,
- $s_{prev(j),j}$ – czas przezbrojenia zależny od kolejności między zadaniami,
- $M_i$ – zbiór zadań przypisanych do maszyny $i$.

Wartość $LB(\pi')$ stanowi oszacowanie najlepszego możliwego $C_{max}$, jakie można osiągnąć po uzupełnieniu bieżącej permutacji $\pi'$ do pełnego harmonogramu.

#### Górne ograniczenie (UB)

Górne ograniczenie $UB_{best}$ reprezentuje wartość $C_{max}$ dla najlepszego dotychczas znalezionego rozwiązania pełnego $\pi$. Nie jest ono liczone w każdym węźle drzewa, lecz aktualizowane tylko po uzyskaniu kompletnego harmonogramu (pełnej permutacji zadań):

$$UB_{best} = \min(UB_{best}, C_{max}(\pi)) \tag{3.2}$$

Dzięki temu ograniczenie górne odzwierciedla rzeczywiste wyniki obliczone dla pełnych rozwiązań, co znacząco redukuje koszt obliczeniowy algorytmu. Węzły są odcinane wyłącznie na podstawie relacji między wartościami dolnego ograniczenia i aktualnego najlepszego rozwiązania:

$$LB(\pi') \geq UB_{best} \Rightarrow \text{gałąź zostaje odcięta.} \tag{3.3}$$

W implementacji przyjęto, że początkowa wartość górnego ograniczenia $UB_{best}$ ustalana jest na podstawie harmonogramu uzyskanego dla *permutacji naturalnej*, czyli kolejności zadań zgodnej z ich indeksem.

#### Parametry i przebieg działania algorytmu

1. **Kryterium stopu:** algorytm działa do momentu pełnego przeszukania przestrzeni rozwiązań i zawsze zwraca **rozwiązanie optymalne**. Gałęzie są odcinane wyłącznie na podstawie relacji między wartościami $LB(\pi')$ i $UB_{best}$.

### 3.2.2. Algorytm metaheurystyczny – Tabu Search

#### Rozwiązanie początkowe

Jako rozwiązanie początkowe przyjęto permutację zadań wygenerowaną za pomocą heurystyki *Minimum Setup Time First (MinSTF)*. Pierwsze zadanie wybierane jest losowo, natomiast każde kolejne $J_j$ dobierane jest w taki sposób, aby minimalizować czas przezbrojenia względem poprzedniego zadania $J_i$:

$$J_{next} = \arg\min_{J_j \notin \pi} s_{i,j}, \tag{3.4}$$

gdzie $s_{i,j}$ oznacza czas przezbrojenia przy przejściu z $J_i$ do $J_j$. Otrzymana sekwencja stanowi rozwiązanie startowe $S_0$.

#### Struktura sąsiedztwa

Zbiór sąsiedztwa $N(S)$ definiowany jest za pomocą pojedynczego typu ruchu:

- **Swap** – zamiana miejscami dwóch zadań $J_i$ oraz $J_j$ w bieżącej permutacji.

W każdej iteracji generowany jest zbiór sąsiadów poprzez wykonanie wszystkich możliwych ruchów typu *swap*, a następnie wybierany jest sąsiad $S'$, który minimalizuje funkcję celu $f(S)$.

#### Lista tabu

W celu uniknięcia cyklicznego odwiedzania tych samych konfiguracji zastosowano **listę tabu (TL)** przechowującą informacje o ostatnich wykonanych ruchach. Długość listy ustalono eksperymentalnie na wartość:

$$L_{tabu} = 10. \tag{3.5}$$

Po wykonaniu ruchu, para zamienionych zadań $(J_i, J_j)$ trafia na listę tabu i pozostaje na niej przez $L_{tabu}$ kolejnych iteracji. – Macerz zamiast listy

#### Kryterium aspiracji

Ruch oznaczony jako tabu może zostać zaakceptowany, jeśli prowadzi do rozwiązania lepszego niż dotychczasowe najlepsze:

$$f(S_{new}) < f(S_{best}). \tag{3.6}$$

Dzięki temu możliwe jest wyjście z lokalnego minimum i poprawa globalnej jakości rozwiązania.

### Parametry i przebieg działania algorytmu

- **Funkcja celu:** minimalizacja całkowitego czasu zakończenia ostatniego zadania w harmonogramie.
- **Heurystyka początkowa:** *MinSTF (Minimum Setup Time First).*
- **Długość listy tabu:** $L_{tabu} = 10$ — lista przechowywana w formie macierzy, co umożliwia sprawdzanie przynależności ruchu w czasie stałym $O(1)$.
- **Liczba iteracji:** $k_{max} = 100$.
- **Kryterium zatrzymania:** brak poprawy przez $\delta = 10$ kolejnych iteracji.

Algorytm rozpoczyna działanie od rozwiązania $S_0$ wygenerowanego metodą *MinSTF*, a następnie iteracyjnie przeszukuje przestrzeń sąsiedztwa poprzez ruchy typu *swap*. Po każdej iteracji aktualizowana jest lista tabu oraz najlepsze znalezione rozwiązanie $S_{best}$. Po spełnieniu kryterium zatrzymania algorytm zwraca harmonogram o najmniejszej wartości $C_{\max}$.

### Dobór parametrów algorytmów

Dobór parametrów poszczególnych algorytmów oparto na analizie dostępnych badań oraz artykułów naukowych stanowiących podstawę opracowanego rozwiązania (omówionych w sekcji 2.2).

## 3.3. Struktura danych wejściowych i wyników optymalizacji

Na podstawie opisanych wcześniej metod optymalizacji (zob. sekcja 2.2) oraz schematu komunikacji (zob. rysunek 3.1) pomiędzy interfejsem graficznym a modułem obliczeniowym w Pythonie, w niniejszej części zaprezentowano strukturę danych wejściowych oraz oczekiwany format wyników generowanych przez silnik optymalizacji.

### Dane wejściowe

Dane wejściowe przekazywane do silnika optymalizacji zapisywane są w formacie `.json`, co zapewnia łatwą integrację z interfejsem graficznym oraz niewielki narzut pamięciowy. Struktura pliku zawiera wszystkie niezbędne informacje do uruchomienia procesu harmonogramowania.

1. **Typ algorytmu** – nazwa metody optymalizacji, która ma zostać użyta (np. `bnb`, `greedy`, `tabu`).
   *Typ:* ciąg znaków (string)

2. **Liczba etapów** – liczba etapów produkcji oznaczona w pracy jako $s$.
   *Typ:* liczba całkowita (int)

3. **Liczba zadań** – liczba zadań do zaplanowania, oznaczona jako $n$ lub $J = \{J_1, J_2, \ldots, J_n\}$.
   *Typ:* liczba całkowita (int)

4. **Liczba maszyn w każdym etapie** – liczność zbioru maszyn w etapach $a = 1, \ldots, s$, oznaczona jako $m_a$ (łącznie $M = \{m_1, m_2, \ldots, m_s\}$).
   *Typ:* tablica liczb całkowitych (array of int)

5. **Współczynnik uczenia się** – oznaczony w pracy jako $\alpha$, określający tempo skracania czasu operacji w zależności od liczby powtórzeń.
   *Typ:* liczba zmiennoprzecinkowa (float)

6. **Etapy uczące się** – binarny ciąg długości $s$, w którym 1 oznacza, że dany etap podlega efektowi uczenia się (np. `"0010"` oznacza, że tylko trzeci etap zawiera etap uczący się).
   *Typ:* ciąg binarny (string)

7. **Czasy operacji** $p_{jak}$ – trójwymiarowa tablica reprezentująca podstawowe czasy przetwarzania zadania $j$ w etapie $a$ na maszynie $k$.
   *Typ:* tensor o wymiarach $n \times s \times m_a$

8. **Czasy przezbrojeń** $s^{ak}_{jj'}$ – czasy przezbrojeń zależne od kolejności zadań $j$ oraz $j'$ na maszynie $k$ w etapie $a$. Wartości te są przechowywane jako czterowymiarowa macierz.
   *Typ:* tensor o wymiarach $n \times n \times s \times m_a$

Dzięki takiej strukturze danych możliwe jest elastyczne definiowanie instancji problemu i ich automatyczne przetwarzanie niezależnie od liczby zadań, etapów i maszyn. Format JSON dodatkowo ułatwia testowanie oraz integrację z różnymi środowiskami programistycznymi.

Poniżej przedstawiono fragment przykładowego pliku wejściowego:

Source code 3.1: Przykładowa struktura pliku `input.json`

```
{
  "algorithm": "greedy",
  "num_stages": 2,
  "num_jobs": 3,
  "machines_per_stage": [1, 2],
  "learning_coefficient": 0.9,
  "learning_stages": "10",

  "processing_times": [
    [[10], [8, 9]],
    [[9], [7, 10]],
    [[11], [6, 8]]
  ],

  "setup_times": [
    [
      [[1], [[3, 2]]],
      [[3], [[2, 3]]],
      [[5], [[4, 5]]]
    ],
    [
      [[3], [[1, 2]]],
      [[2], [[3, 2]]],
      [[4], [[3, 2]]]
    ],
    [
      [[6], [[3, 4]]],
      [[4], [[2, 3]]],
      [[3], [[1, 1]]]
    ]
  ]
}
```

Zawarte w pliku dane opisują przypadek z trzema zadaniami i dwoma etapami, z jedną maszyną przypisaną do pierwszego etapu oraz dwiema maszynami do drugiego etapu. Etap pierwszy jest uczący się (oznaczenie `"learning_stages": "10"`), a czasy operacji oraz przezbrojeń są zapisane w postaci zagnieżdżonych tablic, odpowiednio:

- `processing_times[i][a][k]` – czas operacji $p_{iak}$ dla zadania $i$, etapu $a$, maszyny $k$,
- `setup_times[i][j][a][k]` – czas przezbrojenia $s_{ij}^{ak}$ pomiędzy zadaniami $i \to j$ na etapie $a$ i maszynie $k$. Relacja $i \to i$ oznacza przygotowanie maszyny do produkcji zadania $i$, o wskazuje na konieczność przygotowania maszyny przed rozpoczęciem produkcji, czyli w sytuacji, gdy maszyna nie była wcześniej używana do żadnego zadania.

Taka struktura danych umożliwia łatwą serializację i deserializację, co ułatwia integrację pomiędzy komponentami systemu: silnikiem optymalizacyjnym napisanym w Pythonie oraz interfejsem graficznym w Javie. Format JSON zapewnia przejrzystość, rozszerzalność oraz możliwość szybkiej walidacji danych wejściowych i wyjściowych.

## Wynik działania algorytmu

W wyniku działania algorytmu uzyskiwany jest harmonogram z przypisaniem operacji do maszyn, kolejnością ich realizacji oraz czasami rozpoczęcia i zakończenia, co umożliwia obliczenie $C_j$ oraz $C_{\max}$. Struktura wyjściowa zwracana przez silnik optymalizacyjny jest reprezentowana w formacie JSON, co umożliwia łatwą integrację z interfejsem użytkownika oraz dalsze przetwarzanie danych. Plik wynikowy zawiera listę obiektów reprezentujących zaplanowane akcje na każdej maszynie (operacje i przezbrojenia), wraz z czasem ich trwania.

Każdy wpis zawiera następujące pola:

1. `type` — typ operacji: `"operation"` lub `"setup"`,
2. `job` — numer zadania (dla typu `operation`),
3. `from_job`, `to_job` — zadania źródłowe i docelowe (dla typu `setup`),
4. `stage` — numer etapu produkcji,
5. `machine` — indeks maszyny,
6. `start` — czas rozpoczęcia,
7. `end` — czas zakończenia.

Przykładowa struktura pliku `schedule.json`:

Source code 3.2: Przykładowy fragment pliku wynikowego `schedule.json`

```
  { "type": "setup", "from_job": 1, "to_job": 1, "stage": 1,
      "machine": 1, "start": 0, "end": 1 },
  { "type": "operation", "job": 1, "stage": 1, "machine": 1,
      "start": 1, "end": 10 },
  { "type": "setup", "from_job": 2, "to_job": 2, "stage": 1,
      "machine": 2, "start": 0, "end": 3 },
  { "type": "operation", "job": 2, "stage": 1, "machine": 2,
      "start": 3, "end": 10 },
  { "type": "setup", "from_job": 3, "to_job": 3, "stage": 1,
      "machine": 2, "start": 10, "end": 15 },
  { "type": "operation", "job": 3, "stage": 1, "machine": 2,
      "start": 15, "end": 23 },
  { "type": "setup", "from_job": 1, "to_job": 1, "stage": 2,
      "machine": 1, "start": 0, "end": 3 },
  { "type": "operation", "job": 1, "stage": 2, "machine": 1,
      "start": 3, "end": 13 },
  { "type": "setup", "from_job": 2, "to_job": 2, "stage": 2,
      "machine": 2, "start": 3, "end": 6 },
  { "type": "operation", "job": 2, "stage": 2, "machine": 2,
      "start": 6, "end": 16 },
  { "type": "setup", "from_job": 3, "to_job": 3, "stage": 2,
      "machine": 2, "start": 6, "end": 10 },
  { "type": "operation", "job": 3, "stage": 2, "machine": 2,
      "start": 10, "end": 18 }
]
```

W pliku wynikowym może również znajdować się dodatkowy parametr globalny:

- `"time"` — łączny czas działania algorytmu optymalizacji (czas obliczeń, podany w sekundach),

Przykładowy nagłówek pliku z takim parametrem:

```
{
  "time": 0.317,
  "schedule": [
    ...
  ]
}
```

Dodatkowo, po zakończeniu działania algorytmu generowany jest plik graficzny `gantt_chart.png`, który przedstawia wykres Gantta całego harmonogramu, z rozróżnieniem etapów, maszyn oraz zadań i przezbrojeń. Wykres ten może być automatycznie wyświetlany w GUI lub zapisywany do raportu końcowego.

## 3.4. Projekt Graficznego Interfejsu Użytkownika (GUI)

Graficzny interfejs użytkownika (GUI) aplikacji zostanie zaprojektowany z myślą o prostocie obsługi oraz intuicyjności. Jego głównym celem będzie umożliwienie użytkownikowi wygodnego wczytywania danych wejściowych, definiowania parametrów problemu, uruchamiania algorytmów optymalizacyjnych oraz prezentacji wyników w formie graficznej. Interfejs będzie zbudowany przy użyciu technologii **JavaFX**, co zapewni responsywność oraz możliwość łatwego rozszerzenia w przyszłości.

### Funkcjonalności GUI

1. **Interaktywność** – GUI umożliwi użytkownikowi szybkie przejście między ekranami w celu wczytania danych wejściowych, wyboru algorytmu, uruchomienia obliczeń oraz przeglądania wyników optymalizacji.

2. **Przejrzysta prezentacja wyników** – wyniki algorytmów optymalizacyjnych będą prezentowane w czytelnej formie, zarówno liczbowo, jak i wizualnie. Użytkownik zobaczy harmonogram z dokładnymi czasami operacji na maszynach oraz wykres Gantta, ułatwiający analizę optymalnego rozkładu zadań.

3. **Możliwość eksportu wyników** – po zakończeniu procesu optymalizacji użytkownik będzie miał możliwość eksportu wyników zarówno w formacie JSON, jak i w postaci wykresu Gantta w formacie JPG.

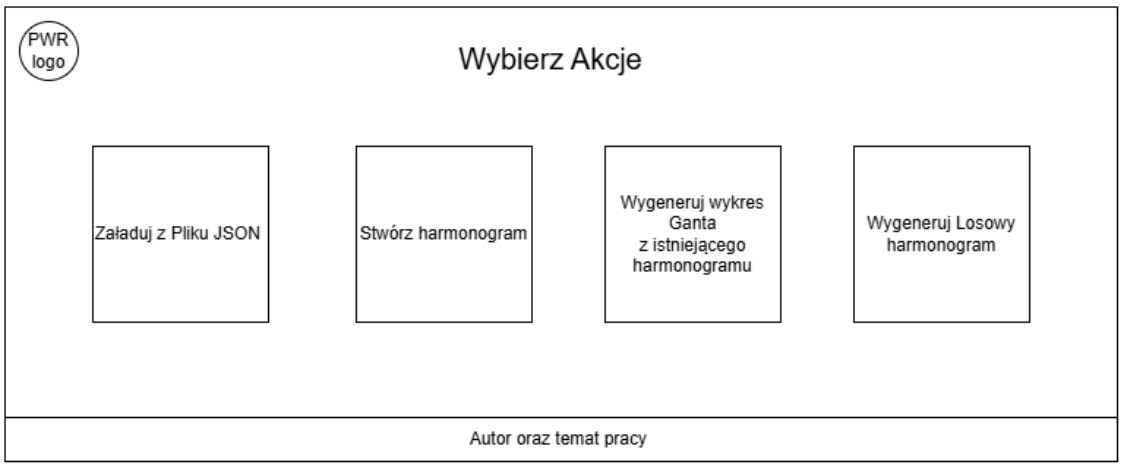
4. **Zarządzanie danymi wejściowymi** – użytkownik będzie mógł wczytywać pliki JSON zawierające dane o zadaniach, maszynach, czasach przetwarzania i przezbrojeniach. Aplikacja umożliwi również ręczne wprowadzenie danych w razie potrzeby.

GUI będzie zaprojektowane w taki sposób, aby każdy etap procesu – od załadowania danych, przez wybór algorytmu, po wizualizację wyników – był jasny i prosty do zrealizowania. Interfejs będzie maksymalnie uproszczony, umożliwiając łatwą nawigację i szybkie przejście do kluczowych funkcji aplikacji.

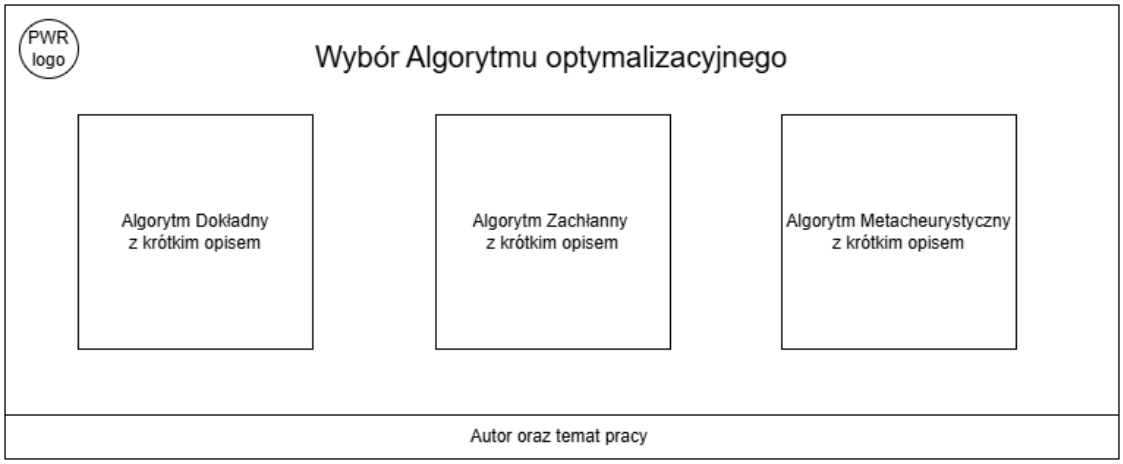
## Szkielet interfejsu użytkownika

Przedstawione poniżej szkielety interfejsu (wireframe) obrazują strukturę oraz układ ekranów aplikacji. Każdy z widoków odpowiada kolejnemu etapowi interakcji użytkownika – od wyboru akcji i algorytmu, poprzez wprowadzanie danych, aż po prezentację wyników działania programu.

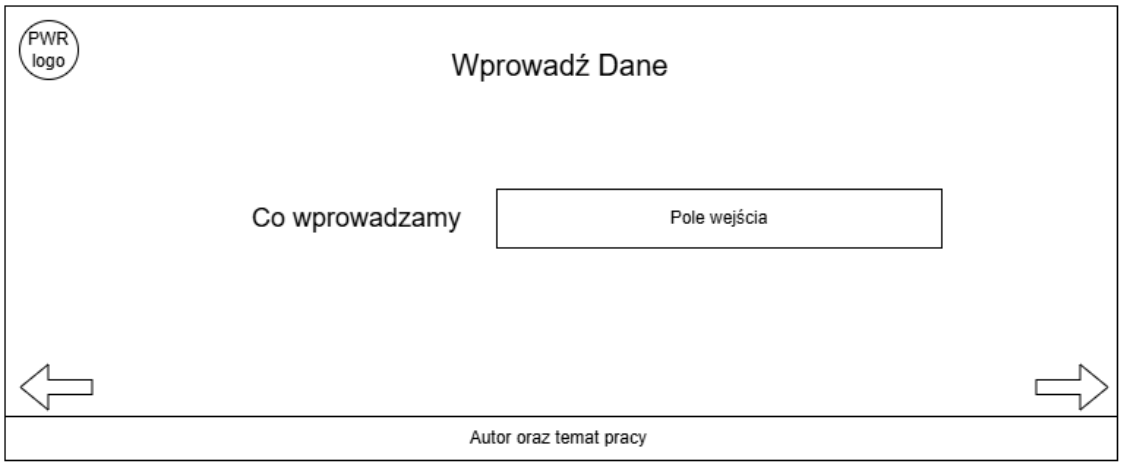
Dane opisujące instancję problemu będą wprowadzane krok po kroku w sposób sekwencyjny. Podczas wykonywania obliczeń aplikacja będzie prezentować nazwę uruchomionego algorytmu oraz szczegółową specyfikację analizowanego przypadku.



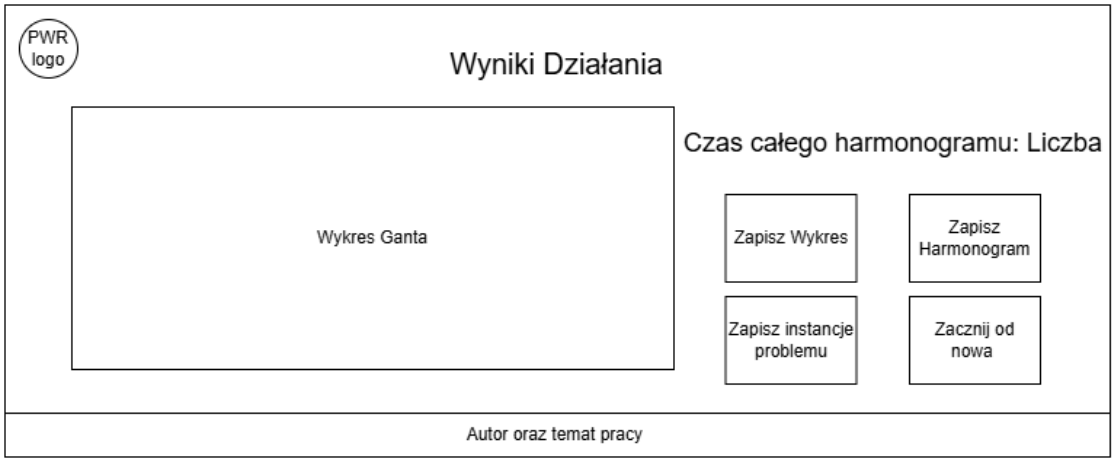

Rysunek 3.2: Widok okna startowego po uruchomieniu aplikacji.




Rysunek 3.3: Widok ekranu wyboru algorytmu optymalizacyjnego.




Rysunek 3.4: Widok ekranu wprowadzania parametrów instancji problemu.




Rysunek 3.5: Widok ekranu z wynikami działania algorytmu.

# 4. Implementacja

Proces implementacji rozpoczęto od modułu obliczeniowego, który stanowi centralny element aplikacji. Jego zadaniem jest realizacja algorytmów optymalizacyjnych, dlatego poprawność i wydajność tej części są kluczowe dla działania całego systemu.

## 4.1. Silnik optymalizacji

W celu zapewnienia uniwersalności kodu oraz możliwości przetwarzania instancji o dowolnej wielkości, w implementacji zastosowano podejście obiektowe. Każda klasa odpowiada za odrębny element modelu problemu, co umożliwia łatwą rozbudowę oraz testowanie poszczególnych modułów systemu.

1. *Job* – klasa reprezentująca pojedyncze zadanie. Przechowuje informacje o czasach przetwarzania operacji na poszczególnych maszynach oraz czasach przezbrojeń pomiędzy zadaniami.

2. *Machine* – klasa opisująca pojedynczą maszynę w systemie produkcyjnym. Zawiera dane o aktualnym stanie maszyny (wolna, w trakcie przezbrojenia, w trakcie przetwarzania), co pozwala na kontrolę dostępności zasobów podczas budowy harmonogramu.

3. *Schedule* – klasa odpowiedzialna za integrację zadań i maszyn w jeden spójny harmonogram. Umożliwia wyznaczenie wartości funkcji celu $C_{max}$ oraz konstrukcję harmonogramu w permutacji naturalnej.

# Bibliografia


[1] H. Harbaoui and S. Khalfallah. Tabu-search optimization approach for no-wait hybrid flow-shop scheduling with dedicated machines. *Procedia Computer Science*, 176:706–712, 2020.

[2] X. Hu, L. Li, and X. Zhao. A branch-and-bound algorithm for project scheduling. *Mathematical Problems in Engineering*, 2015.

[3] A. Kurt. Integrating sequence-dependent setup times and blocking in hybrid flow shop scheduling to minimize total tardiness. *International Journal of Industrial Engineering Computations*, 16(1):147–158, 2025.

[4] C. Liu and W. Zhang. A branch and bound algorithm for the robust parallel machine scheduling problem with sequence-dependent setup time. *arXiv preprint arXiv:1712.00756*, 2017.

[5] L. Pugliese, M. Romano, Y. Lin, W. Zhu, and Y. Tan. Hybrid flow shop scheduling through reinforcement learning: A systematic literature review. *ACM Computing Surveys*, 57(3):1–34, 2025.

[6] I. Ribas, R. Companys, and X. Tort-Martorell. An iterated greedy algorithm for the parallel blocking flow shop scheduling problem with sequence-dependent setup times. *Expert Systems with Applications*, 170:114511, 2021.

[7] W. Yang, S. Li, G. Luo, H. Li, and X. Wen. A real-time human–machine–logistics collaborative scheduling method considering workers' learning and forgetting effects. *Applied System Innovation*, 8(2):40, 2025.

[8] W. Yong. Heuristics for the hybrid flow shop scheduling problem with energy consumption and sequence-dependent setup times. *Mathematical Problems in Engineering*, 2022:1–13, 2022.

[9] Z. Zhang, Z. Shao, W. Shao, J. Chen, and D. Pi. Mrlm: A meta-reinforcement learning-based metaheuristic for hybrid flow-shop scheduling problem with learning and forgetting effects. *Advanced Engineering Informatics*, 60:102445, 2024.

# Spis rysunków

3.1 Diagram komunikacji pomiędzy JavaFX a modułem optymalizacji w Pythonie 14
3.2 Widok okna startowego po uruchomieniu aplikacji. . . . . 22
3.3 Widok ekranu wyboru algorytmu optymalizacyjnego. . . . . 22
3.4 Widok ekranu wprowadzania parametrów instancji problemu. . . . . 23
3.5 Widok ekranu z wynikami działania algorytmu. . . . . 23

# Spis tabel